Implementation of a Knowledge-Based Extractor for Operationalizing Trustworthy AI Requirements

Data Science and its Applications project report for WiSe 2025-26

Faris Abu Ali
faris.abuali@edu.rptu.de

25 March, 2026

Applied Machine Learning, Fachbereich Informatik
RPTU Kaiserslautern
Deutschland

Submitted in partial fulfilment of the requirements for the Degree of M.Sc. Computer Science

Supervised by
Priyabanta Sandulu, M.Sc.
Islam Mesabah, M.Sc.

Abstract

The increasing adoption of artificial intelligence (AI) systems has raised important concerns regarding trustworthiness, regulatory compliance, and responsible deployment. Recent regulatory initiatives such as the European Union Artificial Intelligence Act (EU AI Act) emphasize the need for transparent and accountable AI development processes. However, practitioners often struggle to translate abstract regulatory requirements into actionable engineering practices. This project addresses this challenge by implementing a prototype knowledge extraction pipeline inspired by the conceptual framework proposed in "Actionable Trustworthy AI with a Knowledge-based Debugger" [1]. The system processes unstructured regulatory and guideline documents, extracts structured subject–predicate–object triplets, assesses their novelty relative to an existing knowledge base, and integrates validated knowledge into an expandable knowledge graph under human oversight. The results demonstrate the feasibility of automatically transforming unstructured regulatory documents into structured knowledge representations that can support trustworthy AI development. While the current work focuses on system implementation and qualitative evaluation, it establishes a foundation for future research on automated compliance analysis and AI governance support tools.

Keywords— Structured Knowledge Extraction, Trustworthy AI, AI Governance, Knowledge Graphs, Requirement Engineering, AI Debugger

Contents

1 Introduction **1**

2 Problem statement and Goals **1**
2.1 Goals . . . 1
2.1.1 Goal 1: Transform unstructured regulatory knowledge into structured representations . . . 1
2.1.2 Goal 2: Identify and prioritize relevant knowledge before integration . . . 1
2.1.3 Goal 3: Compare extracted knowledge with an existing knowledge base . . . 2
2.1.4 Goal 4: Combine automated knowledge extraction with human oversight . . . 2

3 Literature Review **2**

4 Implementation Details **3**
4.1 Insights derived from previous work . . . 3
4.2 Technologies used . . . 3
4.3 Student's contribution . . . 3
4.4 Dataset Description and Preprocessing . . . 3
4.5 Pipeline Details . . . 4
4.5.1 Stage 1: Keyword-Guided Knowledge Graph Retrieval . . . 4
4.5.2 Stage 2: Corpus Chunking and Candidate Quality Extraction . . . 5
4.5.3 Stage 3: Subgraph-Conditioned Semantic Filtering . . . 5
4.5.4 Stage 4: Novelty Classification . . . 5
4.5.5 Stages 5–7: Human Oversight, Triplet Extraction, and Knowledge Graph Upsert . . . 6
4.6 Ethical Considerations . . . 7
4.7 Reproducibility and Code Availability . . . 7
4.8 Problems Faced During the Project . . . 8

5 Results **8**

6 Conclusions and Future Scope **11**

List of Figures

1 A flow diagram illustrating the process for extracting, structuring, and integrating trustworthy AI requirements into an expandable knowledge base. Based on the extracted text and processed user input, the system generates new graph pathways, compares them to existing knowledge, and uses human oversight to continuously expand the knowledge base. **Source:** Sandulu et al. [1]. 2

2 **High-level architecture** of the knowledge-based debugger pipeline. The system retrieves existing graph relations related to a user-selected topic (e.g., *explainability*), processes and chunks new corpus documents into paragraphs and then further into sentences, semantically filters these sentences by their relevance to the existing graph relations, evaluates sentence novelty (*Exist*, *New*, *Partially New*), then a human-in-the-loop validates sentences. The system then extracts subject-predicate-object triplets out of the accepted sentences, and integrates them into the knowledge graph. The feedback loop enables continuous knowledge refinement. 4

3 **Stage 1: Keyword-guided knowledge graph retrieval.** The user selects a keyword representing a trustworthy AI requirement. The system queries the Neo4j knowledge graph to retrieve all one-hop relations associated with the keyword, forming the initial context used in later pipeline stages. 4

4 **Stage 2: Corpus chunking and candidate quality extraction.** Input documents are parsed using Docling [2] to obtain structured paragraphs. KeyBERT-based semantic filtering retains paragraphs relevant to the user-selected keyword [3]. An LLM then decomposes these paragraphs into concise, self-contained candidate sentences that will be evaluated in later pipeline stages. 5

5 **Stage 3: Subgraph-conditioned semantic filtering.** Candidate sentences and keyword-matched knowledge graph relations are encoded into a shared embedding space using the `all-MiniLM-L6-v2` SentenceTransformer model [4]. Cosine similarity search identifies the top-k nearest relations for each candidate sentence. A threshold filter retains only sentences sufficiently close to the existing knowledge graph context, while discarded sentences are archived for auditing. 6

6 **Stage 4: Novelty classification.** Each retained candidate sentence is compared with its nearest knowledge graph relations using an LLM-based comparator. The system labels the sentence as EXISTING, PARTIALLY_NEW, or NEW. These labels assist human oversight in deciding whether the information should be incorporated into the knowledge graph. 6

7 **Stages 5–7: Human oversight, triplet extraction, and knowledge graph upsert.** Novelty-labeled candidate sentences are reviewed by a human operator who decides whether to accept or reject the proposed knowledge. Accepted sentences are converted into structured subject–predicate–object triplets using an LLM-based extractor and inserted into the Neo4j knowledge graph together with provenance metadata. . 7

8 **Landing page of the demo** before execution. The interface provides a keyword selector, a document upload component, and a *Run* button. The lower panel contains a *Graph* tab for visualizing retrieved graph context and an *Oversight* tab for reviewing extracted knowledge before insertion into the knowledge graph. 8

9 **Expanded keyword selector** in the prototype interface. The keyword list contains topics derived from the ethical requirements for trustworthy AI defined by the European Commission High-Level Expert Group on AI [5]. In the example run shown in this section, the keyword *"Explainability"* is selected. 9

10 **Retrieved knowledge graph context for the keyword *"Explainability"*.** The displayed one-hop subgraph represents the graph relations associated with the selected keyword currently existing in the knowledge graph. This subgraph serves as the contextual reference for semantic filtering and novelty assessment in later stages. 9

11 **Pipeline execution progress** as shown in the user interface. The system reports intermediate stages including Docling parsing, KeyBERT filtering, LLM-based decomposition, subgraph similarity computation, and novelty classification. The interface is updated through periodic polling, allowing the user to monitor progress until completion. 10

12 **Oversight view interface** after novelty classification. Extracted candidate sentences are grouped into *Existing Knowledge*, *Partial Match*, and *New Knowledge* according to the LLM-based novelty assessment. The human reviewer can inspect the results and select which sentences should proceed to triplet extraction. 10

13 **Triplet review interface** after LLM-based extraction. Each accepted candidate sentence is converted into one or more subject–predicate–object triplets, displayed in editable rows. The reviewer can refine the extracted values before submitting them to the knowledge graph. 11

14 **Updated knowledge graph** after triplet insertion. The interface shows one of the newly added relations together with its metadata, including the originating sentence, source document, and timestamps, supporting provenance tracking and inspection of the inserted knowledge. 11

1 Introduction

Artificial Intelligence (AI) systems are increasingly deployed across a wide range of domains including healthcare, finance, transportation, and public administration. While these systems provide significant benefits, their deployment also raises important concerns regarding transparency, fairness, safety, and regulatory compliance. As a response to these challenges, policymakers and research communities have proposed several governance frameworks aimed at ensuring the development of trustworthy AI systems.

One of the most significant recent regulatory initiatives is the European Union Artificial Intelligence Act (AI Act), which introduces a risk-based regulatory framework for AI systems operating within the European Union [6]. The regulation establishes obligations related to risk management, transparency, human oversight, and data governance. However, translating these high-level legal and ethical requirements into practical engineering procedures remains a challenging task for AI practitioners.

Recent research has highlighted the need for tools that help bridge the gap between abstract regulatory requirements and practical AI development workflows. Sandulu et al. [1] propose the concept of a knowledge-based AI debugger designed to assist practitioners in identifying trustworthiness risks and suggesting actionable mitigation strategies. Their framework introduces a requirement and knowledge engineering pipeline capable of transforming regulatory documents into structured knowledge representations that support automated reasoning and compliance verification. The core idea is to extract structured information from regulatory documents and represent it as subject–predicate–object triplets within a knowledge graph, allowing newly extracted knowledge to be compared against existing knowledge bases and continuously updated through a human-in-the-loop validation process.

Building upon these ideas, this project focuses on implementing the knowledge extraction and integration pipeline required for the knowledge-based debugger framework. The developed system processes unstructured documents, segments them into smaller textual units, extracts structured triplets using large language models, and integrates the resulting knowledge into a graph-based knowledge base.

2 Problem statement and Goals

Over the past few years, the European Union has introduced an increasing number of regulatory, legal, and ethical frameworks aimed at governing the development and deployment of Artificial Intelligence systems. One of the most prominent initiatives is the European Artificial Intelligence Act (AI Act), which defines requirements related to transparency, risk management, human oversight, and accountability for AI systems [6]. While such regulatory frameworks provide important guidance for responsible AI development, they are typically written in natural language and expressed at a high level of abstraction.

For engineers and practitioners who develop and deploy AI systems, this creates a practical challenge. Developers are expected to comply with these regulations, yet the documents themselves do not directly translate into concrete engineering actions. As a result, practitioners must interpret complex legal and ethical documents and manually derive actionable technical requirements from them. This process is time-consuming, error-prone, and requires expertise that many development teams may not possess.

The central problem addressed in this project is therefore the following: how can abstract regulatory knowledge about trustworthy AI be systematically transformed into structured, actionable knowledge that developers can use during the AI development process? The project is primarily practical in nature and focuses on building a working prototype that demonstrates the feasibility of this approach.

2.1 Goals

The objectives of this project are organized into several concrete goals, each addressing a specific challenge involved in transforming unstructured regulatory knowledge into structured and actionable representations.

2.1.1 Goal 1: Transform unstructured regulatory knowledge into structured representations

The first goal of the project is to convert unstructured textual knowledge contained in regulatory and guideline documents into structured representations that can be processed computationally. The project aims to extract structured subject–predicate–object triplets from textual documents. These triplets represent atomic pieces of knowledge that can be integrated into a knowledge graph and linked to existing knowledge structures. By transforming textual requirements into structured representations, developers and practitioners can more easily access relevant rules, constraints, and recommendations that support trustworthy AI development [1].

2.1.2 Goal 2: Identify and prioritize relevant knowledge before integration

Large regulatory documents often contain thousands of paragraphs, many of which may not be directly relevant to a specific topic or engineering context. Extracting knowledge from all available text without filtering can lead to large and noisy knowledge graphs that contain redundant or irrelevant information.

Therefore, the second goal is to identify and prioritize relevant knowledge before integrating it into the knowledge base. The approach consists of finding paragraphs and sentences that are semantically related to specific topics of interest, such as explainability or fairness. To achieve this, several semantic filtering techniques were explored: topic

modelling approaches such as BERTopic [7], keyword extraction techniques such as KeyBERT [3], and vector similarity methods using approximate nearest neighbor (ANN) search algorithms such as FAISS [8] and HNSW [9].

2.1.3 Goal 3: Compare extracted knowledge with an existing knowledge base

The third goal is to implement mechanisms for comparing newly extracted triplets with an existing knowledge base. The system classifies extracted knowledge into three categories: *existing*, *partially new*, and *completely new*, mirroring how humans integrate new information by relating it to previously known concepts. This classification determines whether extracted information should be ignored, updated, or integrated into the knowledge graph. This goal can be seen as an extension of Goal 2.1.2: whereas the previous step prioritizes relevant text segments, this step further filters the extracted knowledge by identifying what is already known, thereby preventing duplication in the knowledge graph.

2.1.4 Goal 4: Combine automated knowledge extraction with human oversight

Although modern NLP techniques and LLMs enable a high degree of automation, fully autonomous decision-making in regulatory interpretation can be risky. Incorrect interpretations of regulatory requirements could lead to incorrect system behavior or compliance violations.

The fourth goal is therefore to maintain a balance between automated knowledge extraction and human oversight. Most pipeline components are automated, including corpus chunking, keyword extraction, similarity search, and classification of knowledge novelty, however, the final decision of accepting or rejecting triplet integration into the knowledge graph remains under human supervision. This human-in-the-loop approach ensures that automated suggestions can be reviewed and corrected when necessary, improving the reliability and trustworthiness of the resulting knowledge base [1].

3 Literature Review

This project builds primarily on two lines of prior work: research on operationalizing trustworthy AI requirements using knowledge engineering pipelines, and research on applying topic modelling techniques to large regulatory texts such as the European AI Act.

The main conceptual foundation for this project is the position paper "Actionable Trustworthy AI with a Knowledge-based Debugger" by Sandulu et al. [1]. The authors propose a framework designed to help practitioners operationalize trustworthy AI requirements by transforming abstract regulatory guidelines into structured and actionable knowledge. As shown in Figure 1, their approach introduces a requirement and knowledge engineering pipeline that extracts information from documents, converts it into subject–predicate–object triplets, and integrates the resulting knowledge into an expandable knowledge base represented as a graph. The framework also proposes an AI debugger that can identify potential risks in AI system components and suggest mitigation strategies, while incorporating human-in-the-loop validation to ensure reliable decision-making. This paper does not present a full implementation of the system but rather outlines the conceptual architecture and motivates the need for such tools in the context of increasingly complex AI governance requirements.

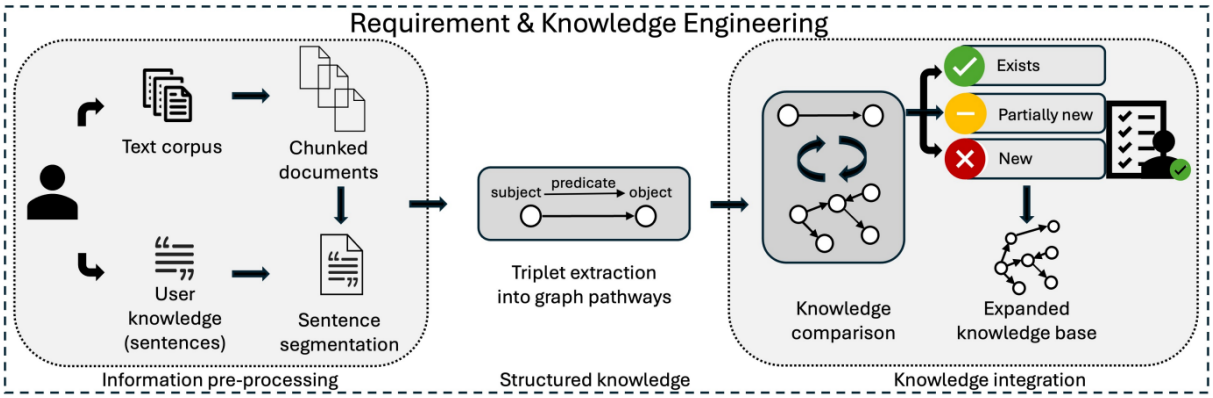

Figure 1: A flow diagram illustrating the process for extracting, structuring, and integrating trustworthy AI requirements into an expandable knowledge base. Based on the extracted text and processed user input, the system generates new graph pathways, compares them to existing knowledge, and uses human oversight to continuously expand the knowledge base. **Source:** Sandulu et al. [1].

A second relevant work is the study by Carbone et al. [10], which explores the application of topic modelling techniques on the legal articles of the AI Act European Directive. The authors investigate methods such as Latent Dirichlet Allocation (LDA) [11], BERTopic [7], and LLM prompting (GPT) to do topic analysis, with the goal of analysing legal requirements using a higher level representation of the underlying topics. Their work could be employed as a guiding tool in the construction of knowledge graphs from the AI Act and other legal directives. Inspired by this

approach, our project explores the use of semantic filtering techniques, including keyword extraction (KeyBERT [3]) and topic-based methods, to identify relevant textual segments before performing knowledge extraction.

Together, these two lines of work inform the methodology developed in this project. From Sandulu et al., we adopt the overall pipeline architecture — triplet extraction, knowledge comparison, and graph-based integration with human oversight. From Carbone et al., we draw inspiration for the semantic filtering stage, applying keyword extraction techniques to identify the most relevant segments of regulatory text before extraction. The following section describes how these ideas are combined and implemented in practice.

4 Implementation Details

4.1 Insights derived from previous work

The architectural design of our system is directly inspired by the requirement and knowledge engineering pipeline proposed by Sandulu et al. [1], illustrated in Figure 1. Our project implements and adapts the core stages of this pipeline: document parsing, semantic filtering, triplet extraction, and knowledge base integration with human-in-the-loop validation.

It is important to note, however, that the framework proposed by Sandulu et al. also includes an AI debugger responsible for identifying potential risks in AI system components and suggesting mitigation strategies. This component is out of scope for the current work. Our focus is exclusively on the structured knowledge extraction and integration pipeline that would feed into such a debugger, leaving the risk mitigation layer as a natural direction for future work.

4.2 Technologies used

The following technologies were selected to support the main stages of the pipeline: document parsing, semantic filtering, keyword extraction, knowledge base integration, and human-in-the-loop validation. The technology choices reflect the specific needs of each stage.

Docling. Docling [2] was used as the document parsing component at the beginning of the pipeline. It processes user-uploaded documents (e.g., PDF, DOC, PPT) and converts them into structured textual segments such as paragraphs. Docling was chosen because it integrates specialized models for document layout analysis (DocLayNet) and table structure recognition (TableFormer), enabling reliable parsing of complex documents such as research papers and legal texts. Its ability to run efficiently on commodity hardware also made it suitable for a lightweight prototype implementation.

Neo4j AuraDB. The project uses Neo4j [12] as the knowledge graph database. Neo4j AuraDB provides a managed graph database environment that supports graph storage, visualization, and querying through the Cypher query language. Neo4j was chosen because of its mature ecosystem, efficient graph traversal capabilities, and its native integration with frameworks such as LangChain, which simplifies interaction between LLM-based pipelines and graph databases.

KeyBERT. KeyBERT [3] is a lightweight keyword extraction technique based on BERT embeddings [13]. It was used to perform keyword-guided semantic filtering on paragraphs extracted from the corpus. Rather than requiring full topic modelling, KeyBERT identifies the most relevant paragraphs with respect to a user-specified keyword (e.g., explainability), making it a lightweight and effective choice for filtering content before downstream processing.

Groq LLM inference. Large language model inference was performed using the Groq API [14]. During development we primarily used the `llama-3.1-8b-instant` model [15], selected for its fast inference speed and generous rate limits. This allowed rapid experimentation with multiple prompt versions, few-shot examples, and pipeline debugging without frequently hitting API constraints.

FAISS and Sentence Transformers. Semantic similarity search was implemented using FAISS (Facebook AI Similarity Search) [8] together with the `sentence-transformers` library [4]. FAISS provides efficient approximate nearest neighbor search over dense vector embeddings, enabling semantic comparison between extracted sentences and existing knowledge graph content. It was chosen over alternatives such as HNSW [9] due to its simpler integration, extensive documentation, and strong performance on the scale of data processed in this project.

Flask. Flask [16] was used as a lightweight web framework to expose the pipeline through a browser-based prototype interface. While not part of the core knowledge extraction architecture, it provided a practical means of interacting with and demonstrating the system during development.

4.3 Student's contribution

This project represents a follow-up implementation effort based on the conceptual architecture proposed by Sandulu et al. [1], carried out by the student under the supervision of Priyabanta Sandulu and Islam Mesabah at DFKI. The student was responsible for the full design and implementation of the prototype pipeline, with the goal of translating the proposed conceptual architecture into a working system.

4.4 Dataset Description and Preprocessing

Since the goal of this project was to develop a prototype pipeline for extracting structured knowledge from documents, the system does not rely on a fixed benchmark dataset. Instead, we used several real-world documents as input corpora to test and validate the behavior of the pipeline during development.

One primary document used during experimentation was the systematic review "Unifying VXAI: A Systematic Review and Framework for the Evaluation of Explainable AI" [17]. This large survey paper contains extensive discussion

on explainable AI methods and evaluation frameworks. It was particularly useful for testing the pipeline with user-selected keywords such as *explainability*, allowing us to evaluate whether the system could successfully retrieve relevant paragraphs and extract meaningful knowledge triplets from a large scientific document.

In addition, we experimented with technical standards such as the IEEE Std 1522-2004: Standard for Testability and Diagnosability Characteristics and Metrics [18]. This type of structured technical document allowed us to observe how the pipeline behaves when processing formal standards and engineering guidelines.

4.5 Pipeline Details

Architecture Overview. Figure 2 presents a high-level view of the knowledge-based debugger pipeline implemented in this project. The system processes a user-selected keyword together with an input document corpus to extract new knowledge and integrate it into the knowledge graph. The architecture follows an iterative knowledge refinement loop, whereby newly extracted knowledge is written back to the graph, allowing future runs of the pipeline to operate on an enriched knowledge base.

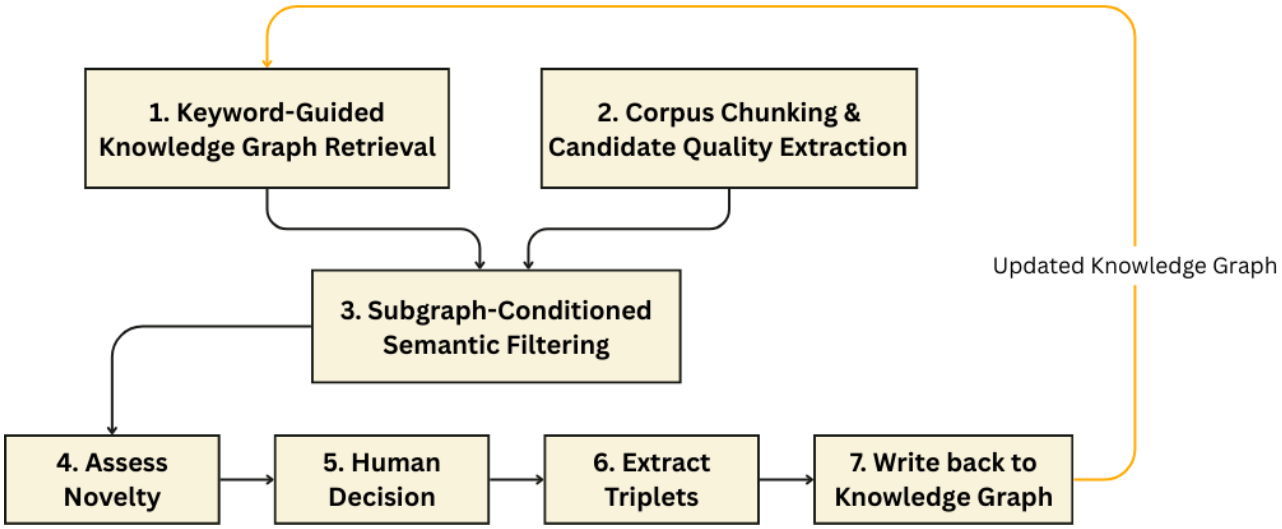

Figure 2: **High-level architecture** of the knowledge-based debugger pipeline. The system retrieves existing graph relations related to a user-selected topic (e.g., *explainability*), processes and chunks new corpus documents into paragraphs and then further into sentences, semantically filters these sentences by their relevance to the existing graph relations, evaluates sentence novelty (*Exist*, *New*, *Partially New*), then a human-in-the-loop validates sentences. The system then extracts subject-predicate-object triplets out of the accepted sentences, and integrates them into the knowledge graph. The feedback loop enables continuous knowledge refinement.

4.5.1 Stage 1: Keyword-Guided Knowledge Graph Retrieval

The pipeline begins with the user selecting a keyword that represents the topic of interest. In the current prototype, the keyword list corresponds to the seven ethical requirements for trustworthy AI defined by the European Commission, including *accountability*, *transparency*, and *fairness* [5].

Using the selected keyword, the system queries the Neo4j knowledge graph to retrieve existing relations (nodes and edges) that contain the keyword in the subject node, object node, or relation predicate. These retrieved relations represent the current knowledge graph context and serve as the reference subgraph for subsequent filtering and novelty assessment stages.

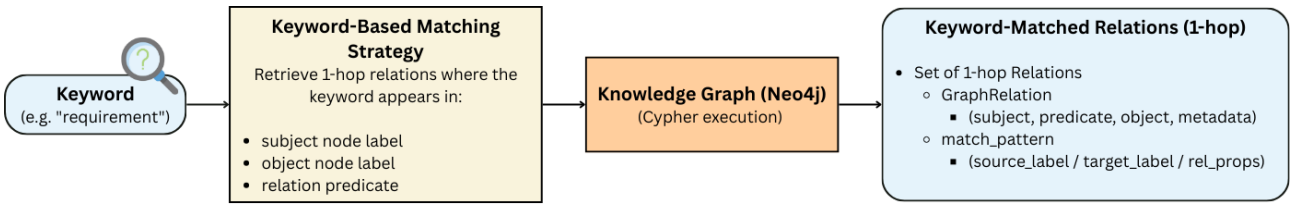

Figure 3: **Stage 1: Keyword-guided knowledge graph retrieval.** The user selects a keyword representing a trustworthy AI requirement. The system queries the Neo4j knowledge graph to retrieve all one-hop relations associated with the keyword, forming the initial context used in later pipeline stages.

4.5.2 Stage 2: Corpus Chunking and Candidate Quality Extraction

At this stage, the input document corpus is parsed and decomposed into smaller textual units that can be analyzed by downstream components. Documents such as PDFs are first processed using Docling [2], which converts them into structured paragraphs while handling layout elements such as tables and document structure.

Next, keyword-guided semantic filtering is applied using KeyBERT [3]. Paragraphs whose extracted keywords match the user-selected topic are retained as relevant candidates. These paragraphs are then passed to an LLM-based decomposition step, where the model decomposes each paragraph into concise, self-contained sentences that are relevant to the selected keyword and will serve as the unit of analysis in subsequent pipeline stages.

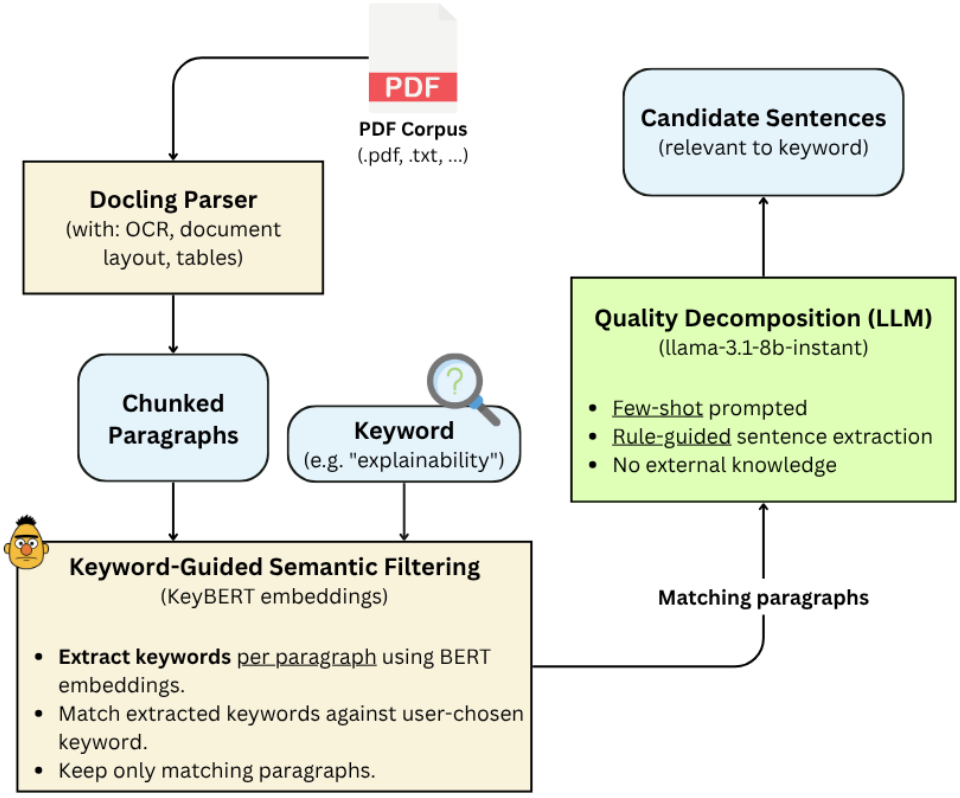

Figure 4: **Stage 2: Corpus chunking and candidate quality extraction.** Input documents are parsed using Docling [2] to obtain structured paragraphs. KeyBERT-based semantic filtering retains paragraphs relevant to the user-selected keyword [3]. An LLM then decomposes these paragraphs into concise, self-contained candidate sentences that will be evaluated in later pipeline stages.

4.5.3 Stage 3: Subgraph-Conditioned Semantic Filtering

At this stage, candidate sentences extracted in Stage 2 are compared against the existing knowledge graph context retrieved in Stage 1. Both the candidate sentences and the keyword-matched graph relations are encoded into the same embedding space using a SentenceTransformer model [4].

The encoded relations of the retrieved one-hop subgraph are treated as a **vector index**, while the candidate sentences act as **query vectors**. Cosine similarity is computed between each candidate sentence and the subgraph relations to obtain the top-k nearest relations ($k \in \{3, 5\}$) for each sentence.

A heuristic threshold filter is then applied: candidates whose maximum similarity score falls below a predefined threshold t_{low} (e.g., 0.55) are discarded, while the remaining sentences are retained together with their nearest graph relations as contextual evidence for later stages. Discarded sentences are archived for auditing purposes. Unlike the paragraph-level filtering performed with KeyBERT in Stage 2 (Section 4.5.2), this stage operates at the sentence level and explicitly conditions the filtering process on the existing knowledge graph structure.

4.5.4 Stage 4: Novelty Classification

After semantic filtering, each retained candidate sentence is evaluated for novelty by comparing it against its nearest knowledge graph relations retrieved in Stage 3. This comparison is performed using an LLM-based novelty comparator, which labels each sentence as one of three categories: EXISTING (no new semantic content), PARTIALLY_NEW (adds a meaningful detail to existing knowledge), or NEW (introduces previously unseen information). The LLM classification is not treated as a final decision but rather as a recommendation to support the human oversight step in the following stage.

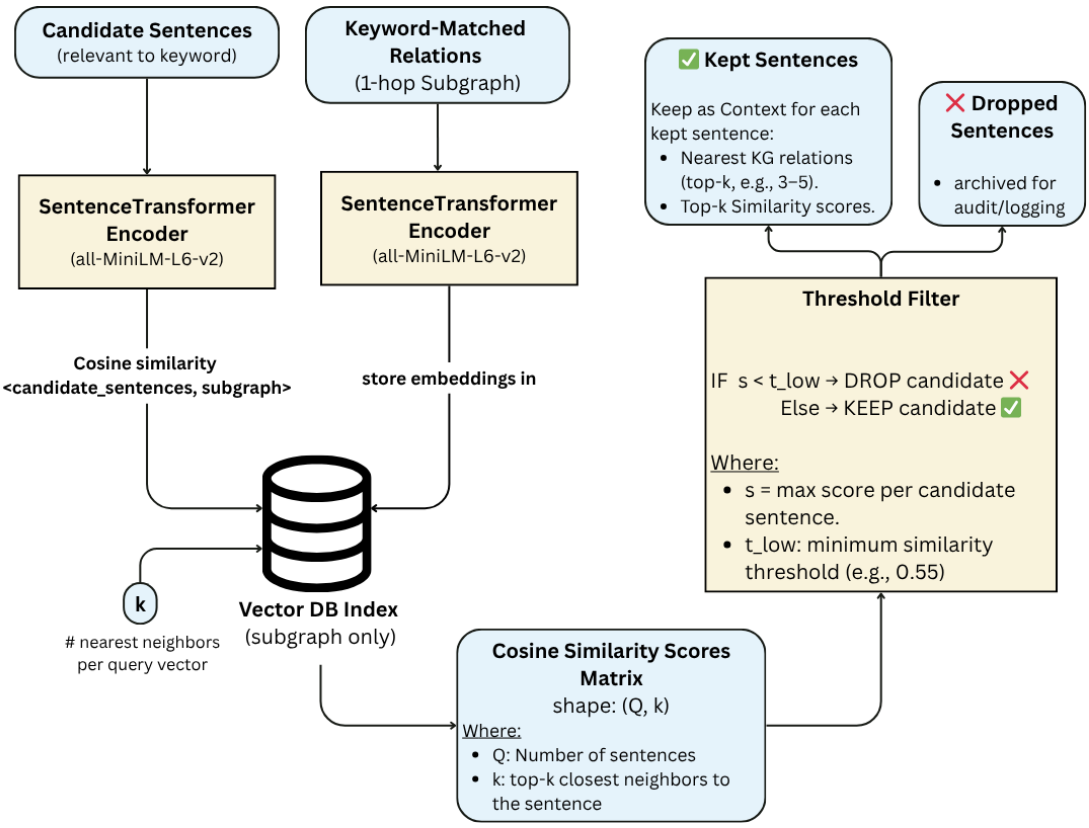

Figure 5: **Stage 3: Subgraph-conditioned semantic filtering.** Candidate sentences and keyword-matched knowledge graph relations are encoded into a shared embedding space using the `all-MiniLM-L6-v2` Sentence-Transformer model [4]. Cosine similarity search identifies the top-k nearest relations for each candidate sentence. A threshold filter retains only sentences sufficiently close to the existing knowledge graph context, while discarded sentences are archived for auditing.

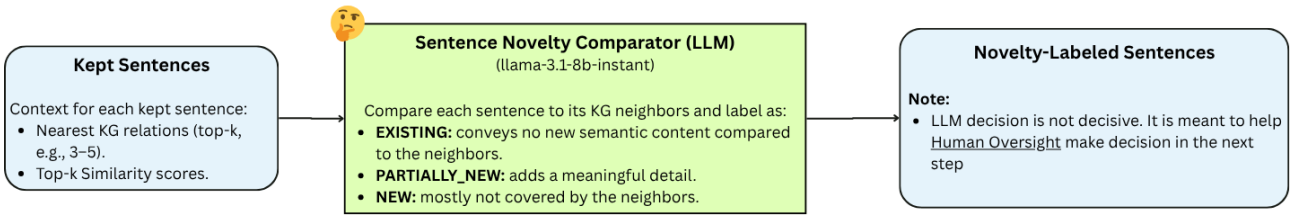

Figure 6: **Stage 4: Novelty classification.** Each retained candidate sentence is compared with its nearest knowledge graph relations using an LLM-based comparator. The system labels the sentence as EXISTING, PARTIALLY_NEW, or NEW. These labels assist human oversight in deciding whether the information should be incorporated into the knowledge graph.

4.5.5 Stages 5–7: Human Oversight, Triplet Extraction, and Knowledge Graph Upsert

In the final stages of the pipeline, the novelty-labeled candidate sentences are presented to a human reviewer. The reviewer decides whether each candidate sentence should be accepted as a valid knowledge contribution or rejected and archived for auditing purposes.

Accepted sentences are then processed by an LLM-based triplet extraction step, which converts each sentence into one or more structured (subject, predicate, object) relations. During extraction, the model resolves pronouns, enforces explicit entity references, and normalizes relation wording to ensure consistency across the knowledge graph. The resulting triplets, together with provenance information such as the originating sentence and source document, are inserted into the Neo4j knowledge graph via an upsert operation. This step completes the iterative knowledge refinement loop illustrated in Figure 2.

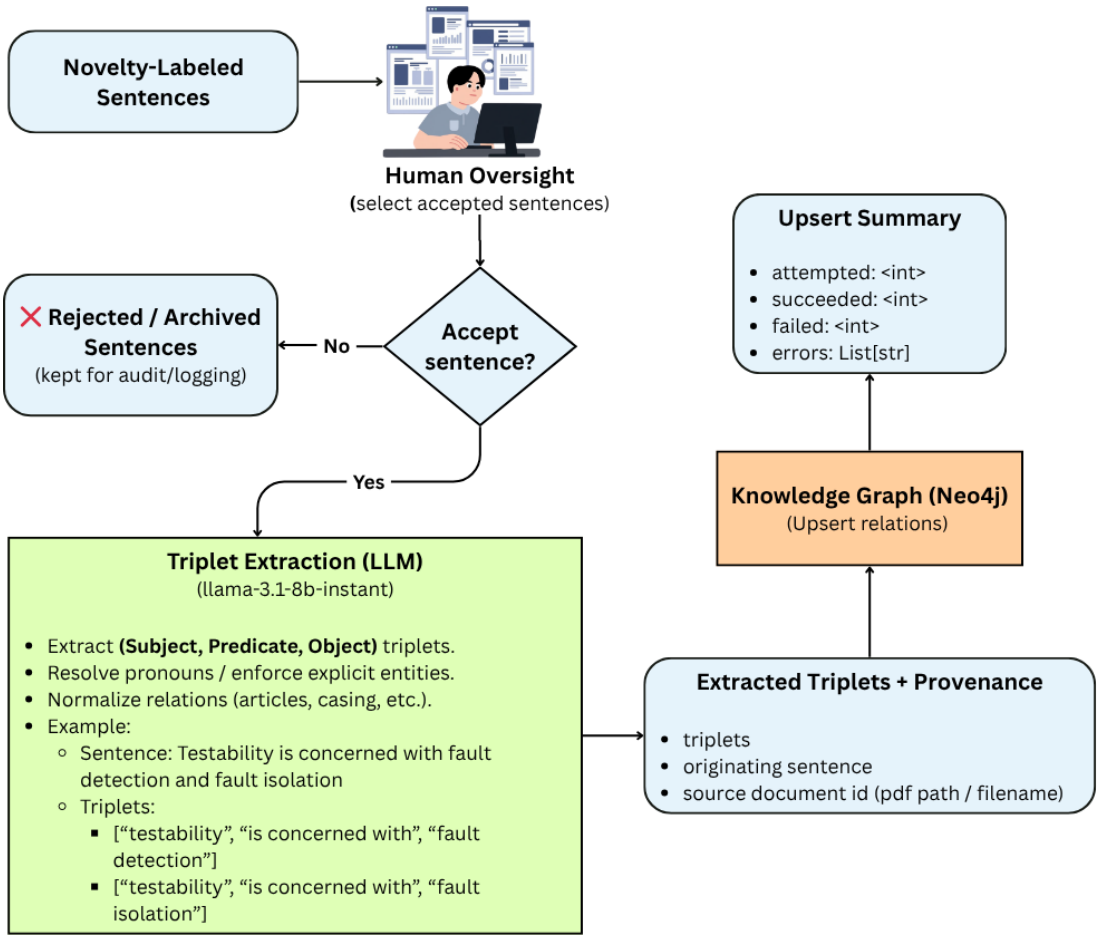

Figure 7: **Stages 5–7: Human oversight, triplet extraction, and knowledge graph upsert.** Novelty-labeled candidate sentences are reviewed by a human operator who decides whether to accept or reject the proposed knowledge. Accepted sentences are converted into structured subject–predicate–object triplets using an LLM-based extractor and inserted into the Neo4j knowledge graph together with provenance metadata.

4.6 Ethical Considerations

The design and deployment of the proposed pipeline raise several ethical considerations worth addressing, even within the scope of a prototype implementation.

Data governance. The system processes external document corpora and stores extracted textual information in a knowledge graph. It is therefore important that all input documents are legally accessible and permitted for such use. During development, only publicly available documents and research papers were used as input corpora. This consideration becomes particularly important in real-world deployments, where organizations must ensure that uploaded documents comply with applicable licensing, copyright, and data governance policies.

External services. The current prototype relies on third-party inference providers for LLM-based operations, including sentence decomposition, novelty assessment, and triplet extraction. In practical deployments, this raises potential concerns related to data privacy and confidentiality, particularly if sensitive or proprietary internal documents are processed through external APIs.

Bias propagation. The pipeline relies on an initial baseline knowledge graph as the starting point for iterative knowledge expansion. Since new knowledge is evaluated relative to the existing graph context, biases or inaccuracies present in the initial knowledge base may influence the direction in which the graph evolves. Without careful oversight, such biases could propagate through successive iterations of knowledge extraction and integration. For this reason, the pipeline explicitly incorporates a human-in-the-loop decision stage before new knowledge is accepted into the graph, ensuring that automated suggestions remain subject to expert validation.

4.7 Reproducibility and Code Availability

Local setup. The source code of the project has been made publicly available on GitHub. The repository contains the full implementation of the pipeline together with instructions for setting up and running the prototype locally.[1]

Live demo. In addition to the local setup, a live demonstration of the system has been deployed as a Hugging Face Space,[2] where the prototype can be accessed directly through the web without requiring local installation. This

[1] `https://github.com/islammesabah/KBExtraction/tree/main/ui`
[2] `https://huggingface.co/spaces/faris-abuali/kbdebugger-demo`

deployment allows users to interact with the system and experiment with the knowledge extraction pipeline in a simplified web-based environment.

4.8 Problems Faced During the Project

Document corpus size and selection. Identifying suitable document corpora for experimentation required more care than initially anticipated. Since the system processes entire documents through multiple stages, very large documents (e.g., 40+ page research papers) significantly increased processing time and made rapid experimentation difficult. As a result, smaller documents were preferred during development to allow faster debugging cycles while still providing meaningful test cases.

Sequential pipeline dependencies. Another challenge arose from the sequential nature of the pipeline architecture. Most stages depend directly on the outputs of previous stages, meaning that debugging later components requires re-running earlier stages of the pipeline. While this design follows naturally from the architecture, it made development iterations slower and required careful logging and inspection of intermediate outputs to efficiently isolate issues.

LLM processing bottlenecks. Computational constraints were observed during LLM-based processing. In the early implementation, the sentence decomposition stage processed each paragraph individually, resulting in hundreds of sequential LLM calls for large documents and long execution times. Attempts to parallelize requests using `ThreadPoolExecutor` were limited by external API rate limits. To mitigate this bottleneck, a batched processing strategy was introduced in which multiple paragraphs are processed within a single LLM call. Combined with the earlier KeyBERT-based filtering stage, this significantly reduced the number of required LLM requests and improved overall pipeline runtime.

5 Results

This section presents results through a sequence of screenshots captured from the running demo. Since the current work focuses on implementing an end-to-end system prototype, the results are primarily qualitative and illustrate how the different stages of the pipeline appear in the user interface during a full run.

Figure 8 shows the landing page of the demo before starting a run. At the top, the user selects a search keyword and uploads an input document. The lower panel contains two tabs: *Graph*, which visualizes the retrieved knowledge graph context for the selected keyword, and *Oversight*, which supports human validation of extracted candidate sentences and triplets. Figure 9 shows the keyword dropdown while expanded.

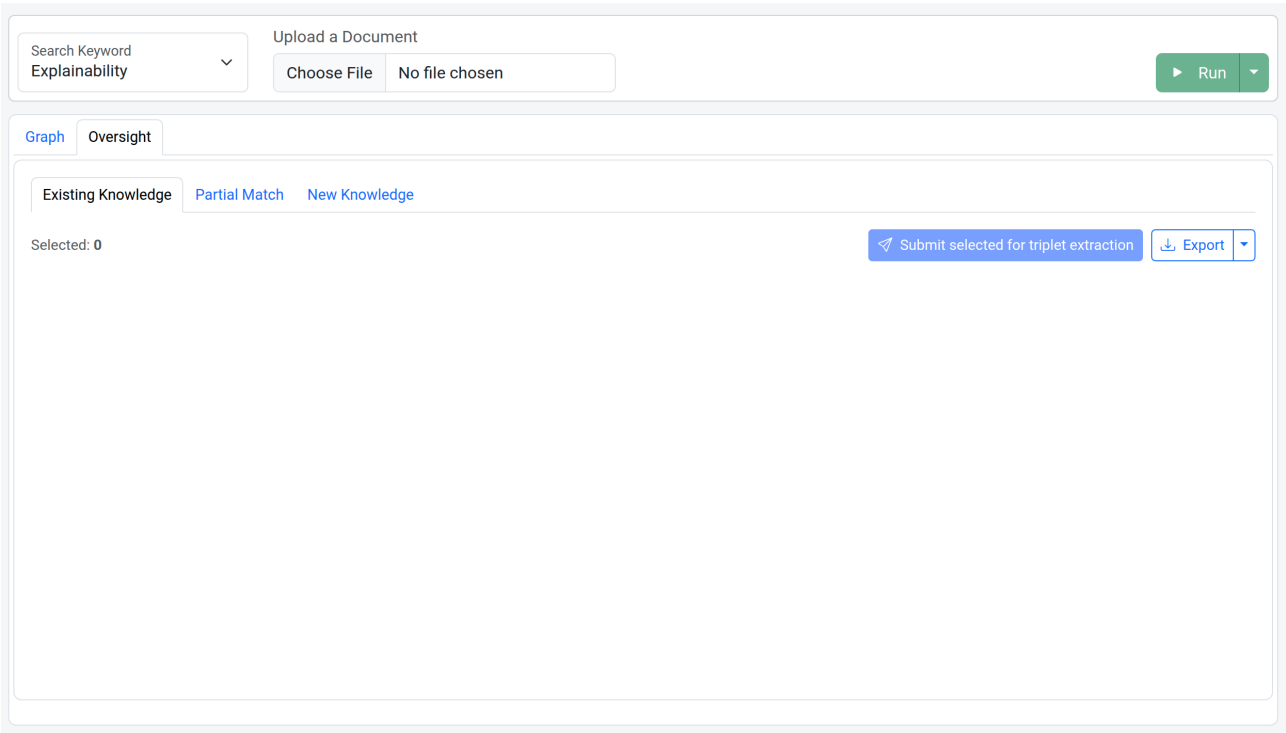

Figure 8: **Landing page of the demo** before execution. The interface provides a keyword selector, a document upload component, and a *Run* button. The lower panel contains a *Graph* tab for visualizing retrieved graph context and an *Oversight* tab for reviewing extracted knowledge before insertion into the knowledge graph.

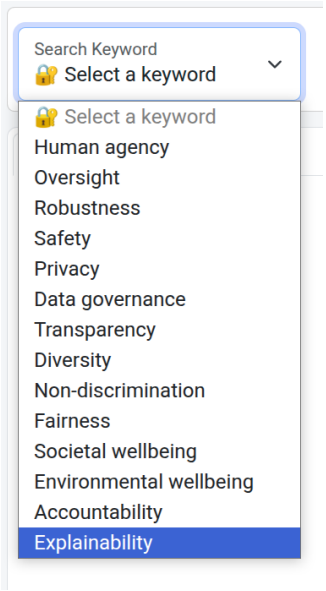

Figure 9: **Expanded keyword selector** in the prototype interface. The keyword list contains topics derived from the ethical requirements for trustworthy AI defined by the European Commission High-Level Expert Group on AI [5]. In the example run shown in this section, the keyword *"Explainability"* is selected.

Subgraph Retrieval. After selecting the keyword, the system retrieves the corresponding one-hop knowledge graph context relevant to the selected keyword. Figure 10 shows the graph view for the keyword *Explainability*. This graph represents the output of Stage 1 (Section 4.5.1), where keyword-guided retrieval is used to collect existing relations from the knowledge graph before processing a new document.

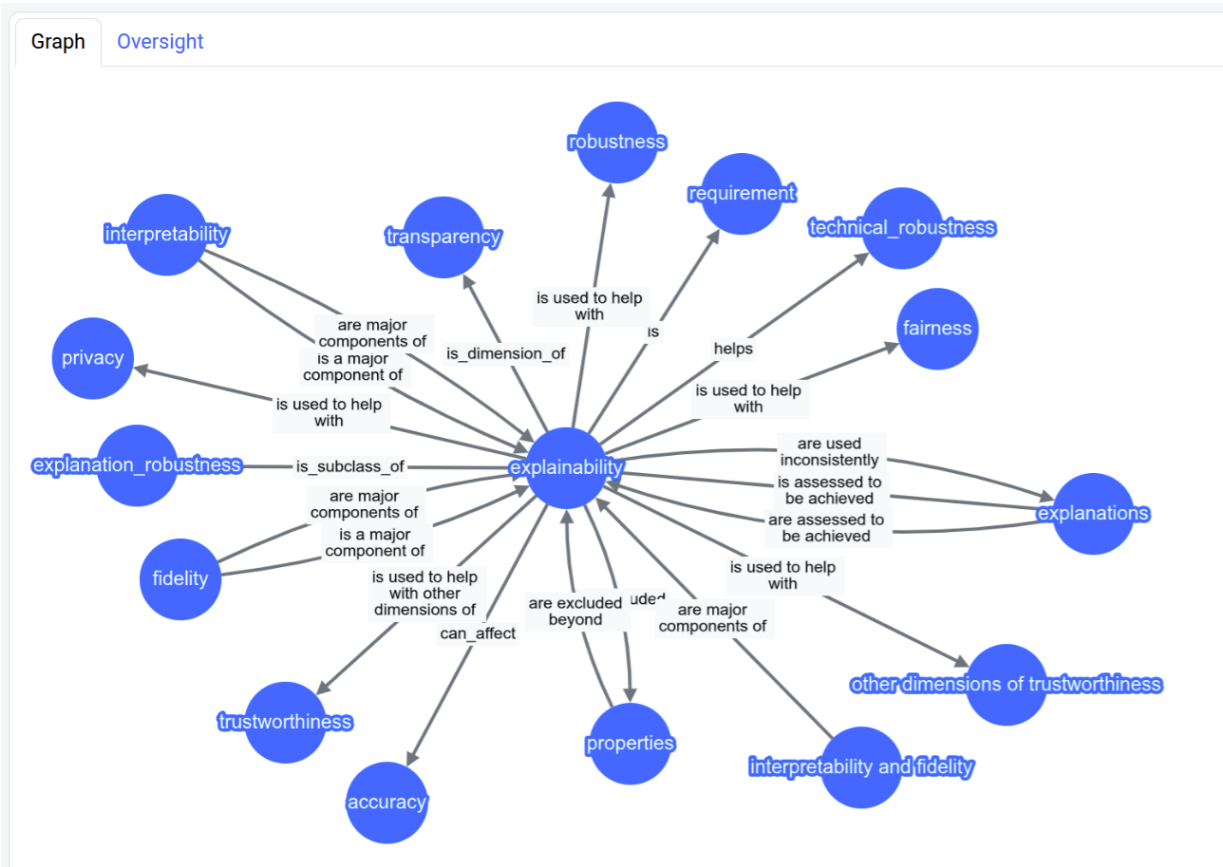

Figure 10: **Retrieved knowledge graph context for the keyword *"Explainability"*.** The displayed one-hop subgraph represents the graph relations associated with the selected keyword currently existing in the knowledge graph. This subgraph serves as the contextual reference for semantic filtering and novelty assessment in later stages.

Document Processing and Intermediate Updates. After retrieving the knowledge graph context, the pipeline is executed on the uploaded document. Since the full pipeline may take several minutes depending on document size, the system continuously polls the backend to provide real-time updates on the current processing stage. Figure 11 shows the sequence of intermediate updates displayed in the user interface while the pipeline is running.

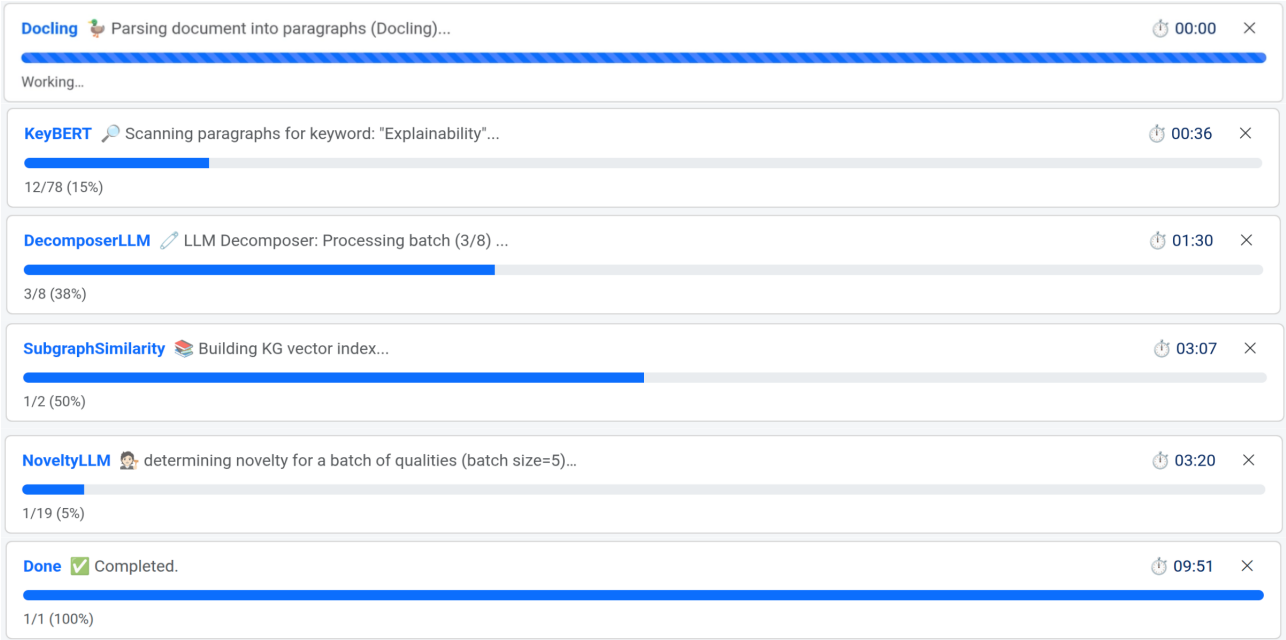

Figure 11: **Pipeline execution progress** as shown in the user interface. The system reports intermediate stages including Docling parsing, KeyBERT filtering, LLM-based decomposition, subgraph similarity computation, and novelty classification. The interface is updated through periodic polling, allowing the user to monitor progress until completion.

Human Oversight. After the pipeline finishes processing, the extracted candidate sentences are displayed in the *Oversight* tab. Figure 12 shows how the system groups the extracted sentences into three categories: *Existing Knowledge*, *Partial Match*, and *New Knowledge*, based on the novelty labels predicted by the LLM. This classification is intended to help the human reviewer validate sentences more efficiently. The final decision remains with the reviewer, who can browse the tabs, inspect the results, and select which sentences should proceed to triplet extraction.

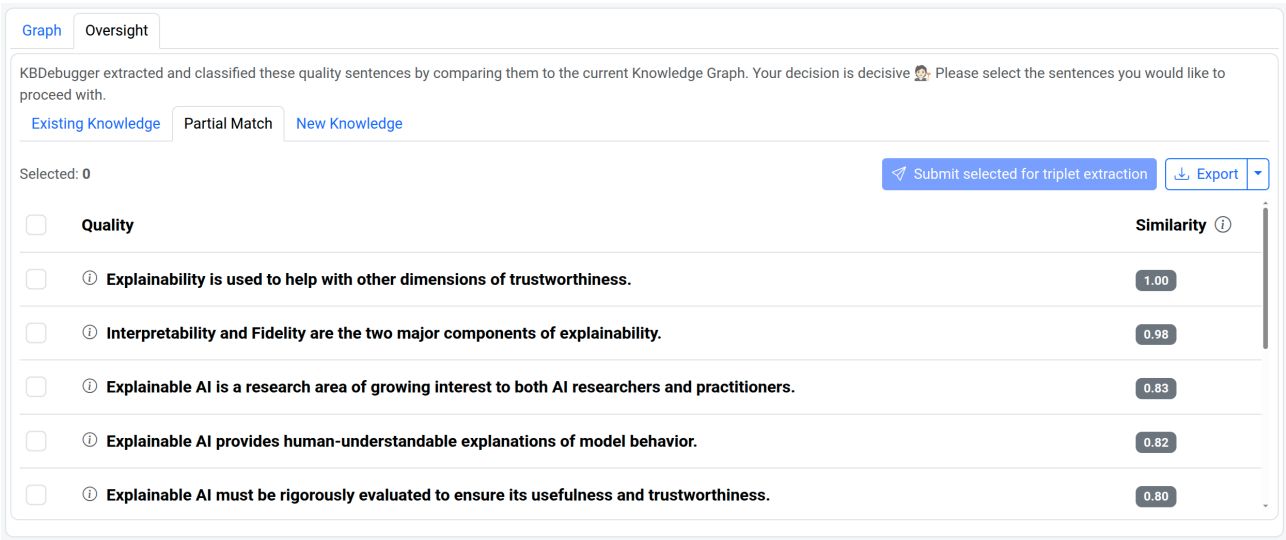

Figure 12: **Oversight view interface** after novelty classification. Extracted candidate sentences are grouped into *Existing Knowledge*, *Partial Match*, and *New Knowledge* according to the LLM-based novelty assessment. The human reviewer can inspect the results and select which sentences should proceed to triplet extraction.

Triplet Review and Refinement. Once the reviewer submits the selected sentences, the pipeline performs triplet extraction and displays the results in a second review table. As shown in Figure 13, each extracted triplet appears in an

editable row containing subject, predicate, and object fields, allowing the reviewer to correct or normalize the extracted relations before inserting them into the knowledge graph.

Knowledge Graph Update. After the reviewed triplets are submitted, the final stage inserts them into the knowledge graph. Figure 14 shows the updated graph together with the details panel of a newly inserted relation. In addition to the relation label, the interface displays metadata such as the originating sentence, source document, and timestamps, supporting traceability and later inspection of the inserted knowledge.

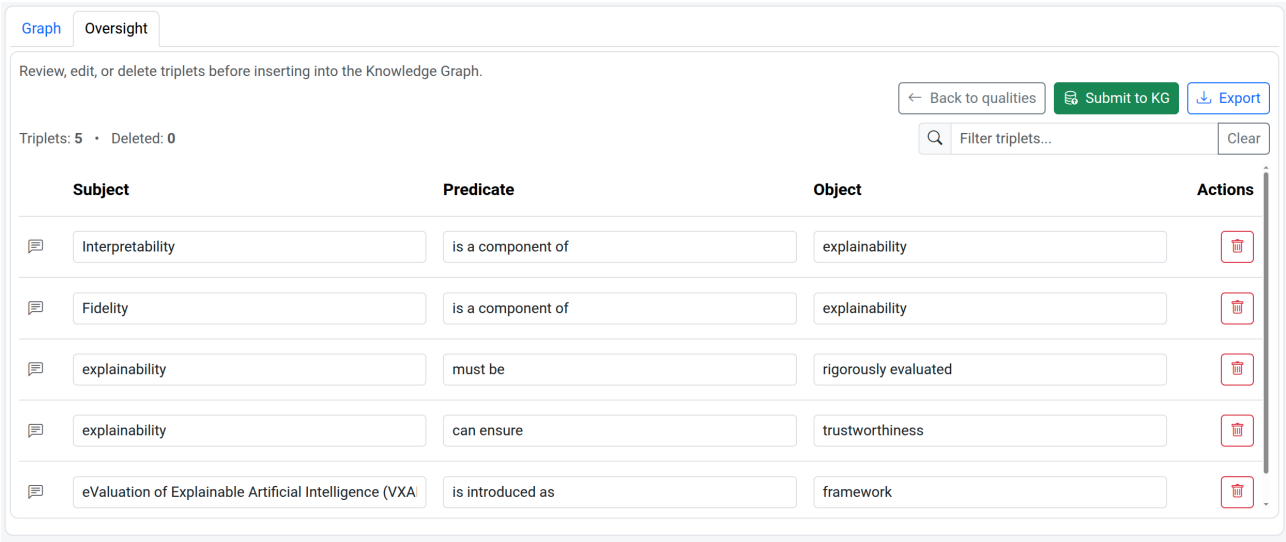

Figure 13: **Triplet review interface** after LLM-based extraction. Each accepted candidate sentence is converted into one or more subject–predicate–object triplets, displayed in editable rows. The reviewer can refine the extracted values before submitting them to the knowledge graph.

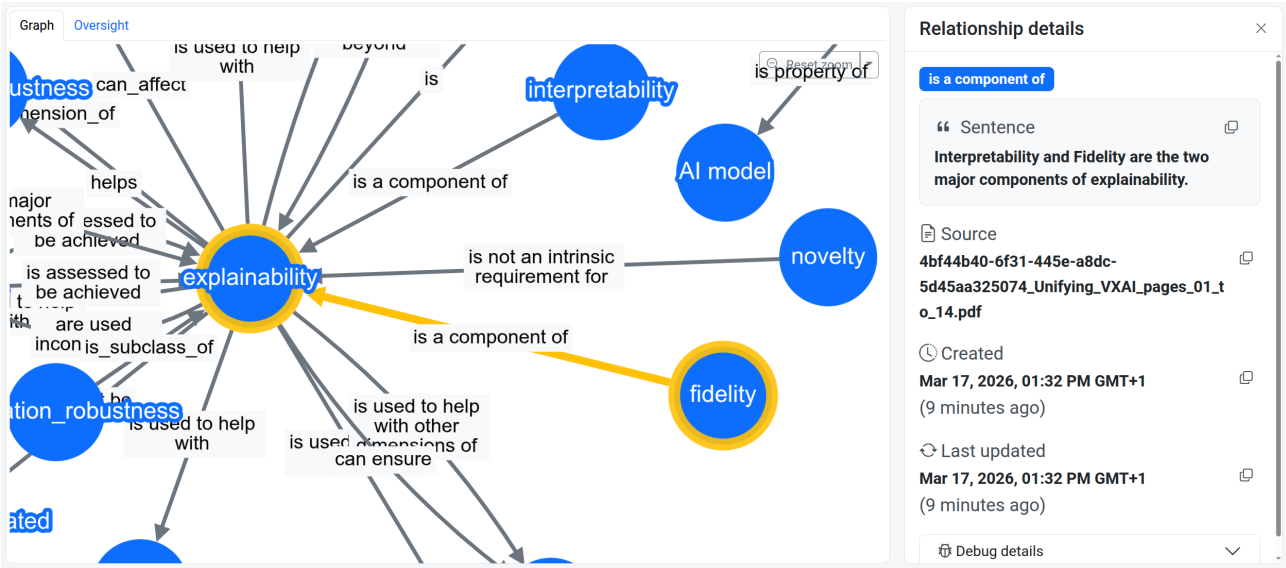

Figure 14: **Updated knowledge graph** after triplet insertion. The interface shows one of the newly added relations together with its metadata, including the originating sentence, source document, and timestamps, supporting provenance tracking and inspection of the inserted knowledge.

6 Conclusions and Future Scope

This project explored the problem of operationalizing abstract trustworthy AI guidelines into structured and actionable knowledge for AI practitioners. Inspired by the architecture proposed by Sandulu et al. [1], the work delivers a prototype pipeline that extracts knowledge from textual corpora and integrates it into an expandable knowledge graph through a complete end-to-end workflow: retrieving existing graph context, processing input documents, filtering and assessing candidate sentences for novelty, and inserting validated knowledge as structured subject–predicate–object triplets. The implementation demonstrates that modern NLP techniques, embedding-based similarity search, and human-in-the-loop

validation can be effectively combined to support the gradual and reliable refinement of a trustworthy AI knowledge base.

Since the primary focus was building a working prototype, evaluation remains qualitative. A natural next step is therefore the development of quantitative evaluation strategies, including benchmarks and metrics for assessing the correctness of extracted triplets and the effectiveness of the filtering and novelty classification stages. On the functional side, the current prototype supports selecting a single keyword per run; extending this to support multi-keyword selection would allow practitioners to explore multiple trustworthy AI dimensions simultaneously, retrieving a unified subgraph across all selected topics in a single pipeline execution. Together, these directions form a clear path toward a more robust system for operationalizing regulatory knowledge in trustworthy AI development.

References

[1] Asbjørn Følstad, Dimitris Apostolou, Steve Taylor, Andrea Palumbo, Eleni Tsalapati, Giannis Stamatellos, and Rosario Catelli. Trust-ai 2025–position paper report. In *Workshop on Trustworthy AI*, 2025.

[2] Nikolaos Livathinos, Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Panos Vagenas, Cesar Berrospi Ramis, Matteo Omenetti, Kasper Dinkla, Yusik Kim, Shubham Gupta, Rafael Teixeira de Lima, Valery Weber, Lucas Morin, Ingmar Meijer, Viktor Kuropiatnyk, and Peter W. J. Staar. Docling: An efficient open-source toolkit for ai-driven document conversion, 2025.

[3] Maarten Grootendorst. Keybert: Minimal keyword extraction with bert., 2020.

[4] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2019.

[5] European Commission High-Level Expert Group on Artificial Intelligence. Ethics guidelines for trustworthy ai. `https://ec.europa.eu/futurium/en/ai-alliance-consultation/guidelines/1.html`, 2019.

[6] European Parliament and Council of the European Union. Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (artificial intelligence act), 2024. ELI: http://data.europa.eu/eli/reg/2024/1689/oj.

[7] Maarten Grootendorst. Bertopic: Neural topic modeling with a class-based tf-idf procedure, 2022.

[8] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library, 2025.

[9] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, 2018.

[10] Ettore Carbone, Alex Giulio Berton, Purbasha Chowdhury, Teresa Scantamburlo, and Paolo Falcarin. Topic modelling on the european ai act. 2025.

[11] David Blei, Andrew Ng, and Michael Jordan. Latent dirichlet allocation. volume 3, pages 601–608, 01 2001.

[12] Emil Eifrem. Introducing Neo4j AuraDB: A new graph database as a service. `https://neo4j.com/blog/auradb/introducing-neo4j-aura-a-new-graph-database-as-a-service/`, November 2019. Accessed: 2024-05-22.

[13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.

[14] Groq, Inc. Groq API: Fast AI inference. `https://groq.com`, 2024. Accessed: 2024.

[15] Abhimanyu Dubey et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[16] Pallets. Flask: The python micro framework for building web applications. `https://palletsprojects.com/projects/flask/`, 2024. Version 3.x, accessed 2024.

[17] David Dembinsky, Adriano Lucieri, Stanislav Frolov, Hiba Najjar, Ko Watanabe, and Andreas Dengel. Unifying vxai: A systematic review and framework for the evaluation of explainable ai. *Transactions on Machine Learning Research (TMLR)*, 1 2026. Survey Certification.

[18] IEEE. IEEE std 1522-2004: IEEE standard for testability and diagnosability characteristics and metrics. IEEE Standard 1522-2004, Institute of Electrical and Electronics Engineers, 2004.